

ML SKILL 1

```
In [1]: import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

1. Dataset Loading

```
In [3]: df = pd.read_csv("student_performance.csv")

df.head()
```

```
Out[3]:
```

	Student_ID	Age	Gender	Study_Hours	Attendance(%)	Test_Score	Grade
0	S0524	21	Female	2.7	74.0	37.0	F
1	S0603	19	Male	1.5	93.1	26.0	F
2	S0527	20	Female	6.8	97.2	88.0	A
3	S0032	18	Female	6.2	93.5	55.0	C
4	S0617	21	Male	7.4	99.2	77.0	B

2. Data Understanding

```
In [4]: print("Shape:", df.shape)

print("\nColumns:")
print(df.columns)

print("\nFirst 5 rows:")
display(df.head())

print("\nStatistical Summary:")
display(df.describe())
```

Shape: (1020, 7)

Columns:

```
Index(['Student_ID', 'Age', 'Gender', 'Study_Hours', 'Attendance(%)',
      'Test_Score', 'Grade'],
      dtype='object')
```

First 5 rows:

	Student_ID	Age	Gender	Study_Hours	Attendance(%)	Test_Score	Grade
0	S0524	21	Female	2.7	74.0	37.0	F
1	S0603	19	Male	1.5	93.1	26.0	F
2	S0527	20	Female	6.8	97.2	88.0	A
3	S0032	18	Female	6.2	93.5	55.0	C
4	S0617	21	Male	7.4	99.2	77.0	B

Statistical Summary:

	Age	Study_Hours	Attendance(%)	Test_Score
count	1020.000000	989.000000	990.000000	1020.000000
mean	20.970588	5.160768	84.871919	49.843137
std	2.008109	2.555877	10.007239	21.278630
min	18.000000	-2.000000	50.300000	0.000000
25%	19.000000	3.700000	78.000000	35.000000
50%	21.000000	5.100000	85.000000	50.000000
75%	23.000000	6.300000	91.975000	64.000000
max	24.000000	25.000000	115.600000	100.000000

3. Data Types

In [5]: `df.dtypes`

```
Out[5]: Student_ID      object
Age                int64
Gender             object
Study_Hours        float64
Attendance(%)       float64
Test_Score          float64
Grade              object
dtype: object
```

4. Missing-Value Detection

In [7]: `df.isnull().sum()`

```
Out[7]: Student_ID      0
        Age            0
        Gender         0
        Study_Hours    31
        Attendance(%)  30
        Test_Score     0
        Grade          9
        dtype: int64
```

```
In [8]: print("Total missing values:", df.isnull().sum().sum())
```

Total missing values: 70

```
In [9]: numeric_columns = ['Age', 'Study_Hours', 'Attendance(%)', 'Test_Score']

for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())
```

5. Missing-Value Handling

```
In [10]: categorical_columns = ['Gender', 'Grade']

for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```
In [11]: df.isnull().sum()
```

```
Out[11]: Student_ID      0
        Age            0
        Gender         0
        Study_Hours    0
        Attendance(%)  0
        Test_Score     0
        Grade          0
        dtype: int64
```

6. Duplicate Detection and Removal

```
In [12]: print("Number of duplicate rows:", df.duplicated().sum())
```

Number of duplicate rows: 20

```
In [13]: df = df.drop_duplicates()

print("Shape after removing duplicates:", df.shape)
```

Shape after removing duplicates: (1000, 7)

7. Categorical-Data Cleaning

```
In [14]: print("Gender values:")
        print(df['Gender'].unique())
```

```
print("\nGrade values:")
print(df['Grade'].unique())
```

Gender values:
['Female' 'Male']

Grade values:
['F' 'A' 'C' 'B' 'A+']

```
In [15]: df['Gender'] = df['Gender'].str.strip().str.title()
df['Grade'] = df['Grade'].str.strip().str.upper()
```

```
In [16]: print(df['Gender'].unique())
print(df['Grade'].unique())
```

['Female' 'Male']
['F' 'A' 'C' 'B' 'A+']

8. Logical-Value Validation

```
In [17]: print(df[df['Age'] < 15])
print(df[df['Age'] > 100])
```

Empty DataFrame

Columns: [Student_ID, Age, Gender, Study_Hours, Attendance(%), Test_Score, Grade]

Index: []

Empty DataFrame

Columns: [Student_ID, Age, Gender, Study_Hours, Attendance(%), Test_Score, Grade]

Index: []

```
In [18]: print(df[df['Study_Hours'] < 0])
```

	Student_ID	Age	Gender	Study_Hours	Attendance(%)	Test_Score	Grade
157	S0045	20	Male	-2.0	100.9	4.0	F
619	S0689	18	Female	-0.5	82.2	6.0	F
775	S0497	18	Male	-0.8	77.9	0.0	F
909	S0725	24	Male	-0.7	58.2	0.0	F

```
In [19]: print(df[(df['Attendance(%)'] < 0) | (df['Attendance(%)'] > 100)])
```

	Student_ID	Age	Gender	Study_Hours	Attendance(%)	Test_Score	Grade
9	S0412	22	Male	5.1	104.8	49.0	F
63	S0463	20	Female	9.3	103.0	100.0	A+
88	S0714	22	Male	5.3	107.2	51.0	C
103	S0347	21	Male	4.9	106.9	55.0	C
134	S0311	24	Female	6.4	101.7	45.0	F
...	...	...	...	...	...	...	...
997	S0770	24	Female	10.3	100.1	86.0	A
1003	S0100	24	Female	0.9	108.3	14.0	F
1016	S0271	22	Male	5.3	100.5	45.0	F
1017	S0861	21	Female	4.0	102.7	30.0	F
1019	S0103	21	Female	1.6	104.6	37.0	F

[61 rows x 7 columns]

9. Categorical Encoding

```
In [20]: df['Gender'] = df['Gender'].map({
        'Female': 0,
        'Male': 1
    })
```

```
In [21]: label_encoder = LabelEncoder()

df['Grade_Encoded'] = label_encoder.fit_transform(df['Grade'])
```

```
In [22]: print(dict(zip(
        label_encoder.classes_,
        label_encoder.transform(label_encoder.classes_)
    )))
```

```
{'A': np.int64(0), 'A+': np.int64(1), 'B': np.int64(2), 'C': np.int64(3), 'F': np.int64(4)}
```

10. Feature Selection

```
In [23]: X = df[['Age', 'Gender', 'Study_Hours', 'Attendance(%)', 'Test_Score']]
```

```
In [24]: y = df['Grade_Encoded']
```

```
In [25]: print("Features:")
display(X.head())

print("\nTarget:")
display(y.head())
```

Features:

	Age	Gender	Study_Hours	Attendance(%)	Test_Score
0	21	0	2.7	74.0	37.0
1	19	1	1.5	93.1	26.0
2	20	0	6.8	97.2	88.0
3	18	0	6.2	93.5	55.0
4	21	1	7.4	99.2	77.0

Target:

```
0    4
1    4
2    0
3    3
4    2
```

Name: Grade_Encoded, dtype: int64

11. Train-Test Split

```
In [26]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42,
        stratify=y
    )
```

12. Standardization

```
In [27]: standard_scaler = StandardScaler()

X_train_standardized = standard_scaler.fit_transform(X_train)
X_test_standardized = standard_scaler.transform(X_test)
```

13. Min-Max Normalization

```
In [29]: minmax_scaler = MinMaxScaler()

X_train_normalized = minmax_scaler.fit_transform(X_train)
X_test_normalized = minmax_scaler.transform(X_test)
```

```
In [30]: print("Standardized data:")
        print(X_train_standardized[:5])

        print("\nMin-Max normalized data:")
        print(X_train_normalized[:5])
```

Standardized data:

```
[[ 1.0308699 -1.04344279 -0.83458162  0.19565016 -0.17468189]
 [-0.48325969 -1.04344279  0.06112156 -1.09756532  0.10583771]
 [-0.98796956  0.95836592 -0.95672296 -1.07703809 -0.73572109]
 [ 0.52616003  0.95836592  1.40467634 -0.99492917  1.32142263]
 [-0.98796956  0.95836592 -0.020306   0.19565016  0.10583771]]
```

Min-Max normalized data:

```
[[0.83333333 0.         0.18888889 0.54545455 0.46         ]
 [0.33333333 0.         0.27037037 0.34796238 0.52         ]
 [0.16666667 1.         0.17777778 0.35109718 0.34         ]
 [0.66666667 1.         0.39259259 0.36363636 0.78         ]
 [0.16666667 1.         0.26296296 0.54545455 0.52         ]]
```

14. fit(), transform(), and fit_transform()

```
In [40]: scaler = StandardScaler()

        # fit()
```

```

scaler.fit(X_train)

# transform()
X_train_transformed = scaler.transform(X_train)

# fit_transform()
X_train_fit_transformed = scaler.fit_transform(X_train)

```

```

In [41]: print("Preprocessing completed successfully.")
print("Final dataset shape:", df.shape)

print("\nFinal dataset:")
display(df.head())

print("\nMissing values after preprocessing:")
print(df.isnull().sum())

print("\nDuplicate rows after preprocessing:")
print(df.duplicated().sum())

```

Preprocessing completed successfully.

Final dataset shape: (1000, 8)

Final dataset:

	Student_ID	Age	Gender	Study_Hours	Attendance(%)	Test_Score	Grade	Grade_Encoded
0	S0524	21	0	2.7	74.0	37.0	F	4
1	S0603	19	1	1.5	93.1	26.0	F	4
2	S0527	20	0	6.8	97.2	88.0	A	0
3	S0032	18	0	6.2	93.5	55.0	C	3
4	S0617	21	1	7.4	99.2	77.0	B	3

Missing values after preprocessing:

```

Student_ID      0
Age              0
Gender           0
Study_Hours      0
Attendance(%)    0
Test_Score       0
Grade            0
Grade_Encoded    0
dtype: int64

```

Duplicate rows after preprocessing:

```
0
```